

Build a Production-Ready CI/CD Pipeline on AWS

Project Documentation

[Docker](#) • [Amazon ECR](#) • [ECS Fargate](#) • [GitHub Actions](#)

Overview

A fast-growing food delivery startup is rapidly adding new features to its backend services to support menus, pricing updates, and promotions. The engineering team currently:

- Builds and deploys applications **manually**
- Runs updates directly on production servers
- Faces **downtime during each deployment**
- Experiences inconsistent environments across releases

As the platform scales, this manual approach creates serious problems:

- Slower release cycles that delay product improvements
- Deployment errors caused by human intervention
- Difficulty rolling out fixes quickly when issues arise
- Increased operational risk with every release

To address these issues, the team will adopt a modern DevOps approach using containerization and automated CI/CD pipelines on AWS.

Solution

The goal is to design and implement a fully automated CI/CD pipeline that takes application code from development to production without downtime.

The solution uses the following AWS and DevOps tools:

- **Docker** - containerizes the application for consistent, portable deployments
- **Amazon ECR** - stores container images securely in a managed registry
- **Amazon ECS Fargate** - runs containers serverlessly without managing servers
- **Application Load Balancer** - distributes traffic and enables zero-downtime deployments
- **GitHub Actions** - automates the full build-push-deploy pipeline on every code push

Every code change pushed to GitHub automatically triggers the pipeline, which:

- Builds a new container image from the latest code
- Pushes the image to Amazon ECR
- Deploys the updated container to ECS Fargate

This delivers consistent deployments, faster releases, and production-grade reliability.

Steps to Build the Pipeline

This guide walks through six phases to build the complete CI/CD pipeline:

- **Understand and run the application locally**
- **Containerize the application using Docker**
- **Push the container image to Amazon ECR**
- **Deploy the containerized app on ECS Fargate**
- **Configure load balancing and security groups**
- **Automate deployments using GitHub Actions**

About the Application

The application is a Food Delivery Menu Service representative of the kind of backend used in real food-delivery platforms. It is built with Node.js and Express and includes:

- **Backend API** - Express server that exposes menu data, health checks, and version endpoints
- **Static Frontend UI** - served directly by the backend; used to visually verify deployments

Source Code: <https://github.com/miloplacid7-cloud/food-menu-CI-CD>

Application Components

1. Backend API Layer

The backend runs an HTTP server and exposes three key endpoints:

- `/api/menu` - Returns the full list of food menu items
- `/health` - Used by the load balancer to check service health
- `/version` - Displays the currently deployed application version

2. Frontend Static UI

The frontend is a lightweight static web interface that:

- Displays the food menu in the browser
- Fetches data from the backend API
- Shows the currently deployed application version

The UI is used to visually confirm successful deployments, mirroring how real teams verify production releases.

Project Folder Structure

The project follows a simple, flat structure:

```
food-menu-service/  
├─ public/  
│   └─ index.html  
├─ server.js  
└─ package.json
```

- `server.js` - Application entry point; configures Express, defines routes, starts the server
 - `public/index.html` - Static frontend UI served to browser clients
 - `package.json` - Lists dependencies and defines the start script
-

Phase 1: Understand and Run the Application Locally

Before containerizing anything, confirm that the application runs correctly on your local machine. This validates the code and helps you understand the project structure before moving to AWS.

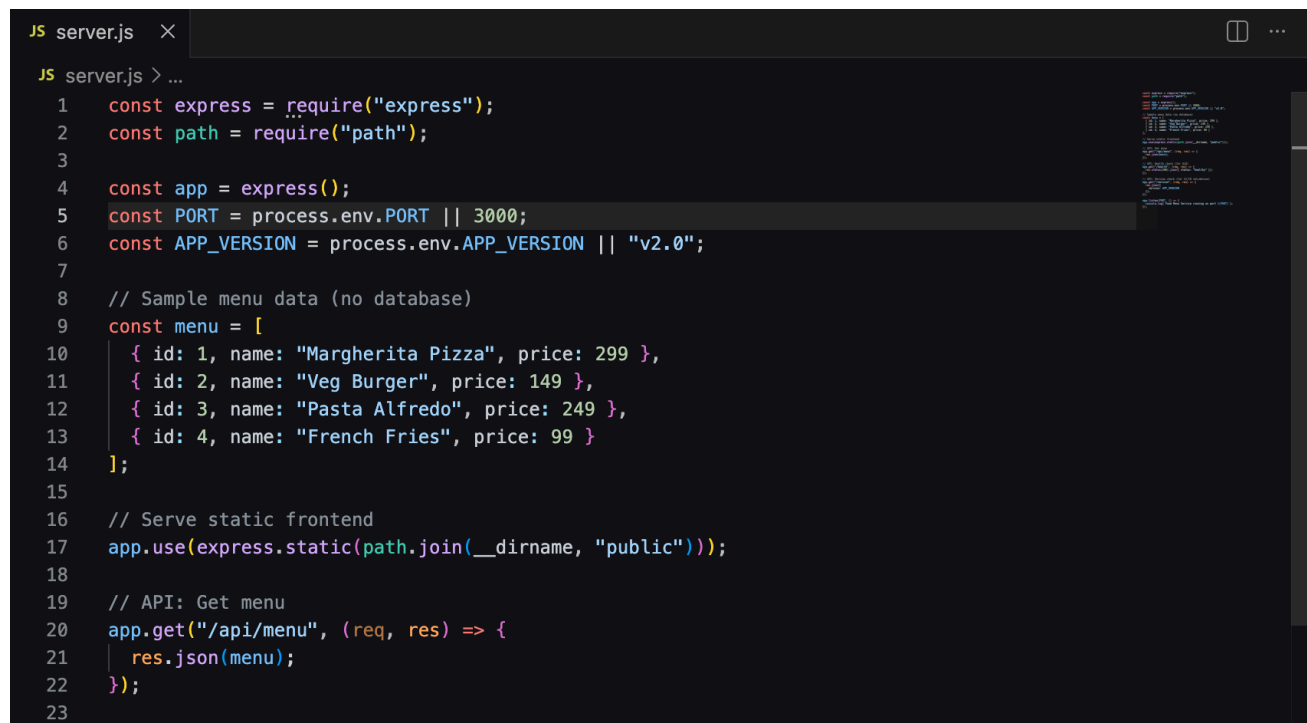
Step 1: Identify the Application Entry Point

Open `server.js`. This file configures the Express server, serves the static frontend, defines all API routes, and starts the application on a specific port. It will be the core file referenced throughout containerization and deployment.

Step 2: Identify the Application Port

Inside `server.js`, the application listens on port 3000, defined with a fallback:

```
process.env.PORT || 3000
```



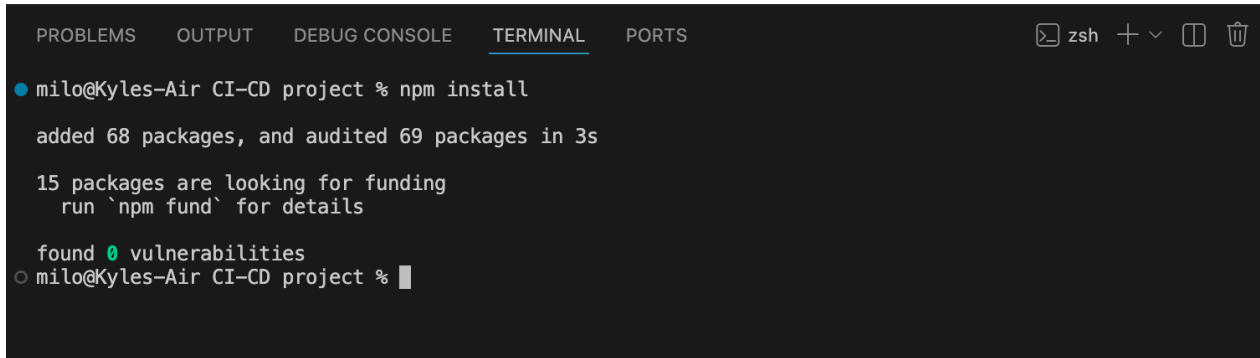
```
JS server.js  X
JS server.js > ...
1  const express = require("express");
2  const path = require("path");
3
4  const app = express();
5  const PORT = process.env.PORT || 3000;
6  const APP_VERSION = process.env.APP_VERSION || "v2.0";
7
8  // Sample menu data (no database)
9  const menu = [
10   { id: 1, name: "Margherita Pizza", price: 299 },
11   { id: 2, name: "Veg Burger", price: 149 },
12   { id: 3, name: "Pasta Alfredo", price: 249 },
13   { id: 4, name: "French Fries", price: 99 }
14 ];
15
16 // Serve static frontend
17 app.use(express.static(path.join(__dirname, "public")));
18
19 // API: Get menu
20 app.get("/api/menu", (req, res) => {
21   res.json(menu);
22 });
23
```

This port will later be **exposed in the Docker container**, configured in the ECS task definition, and used by the Application Load Balancer.

Step 3: Install Application Dependencies

From the project root, install the required packages:

```
npm install
```

A screenshot of a terminal window with a dark background. The terminal shows the output of the 'npm install' command. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected), and 'PORTS'. On the right side of the terminal, there are icons for a shell (zsh), a plus sign, a dropdown arrow, a window icon, and a trash can icon. The output text in the terminal is: '● milo@Kyles-Air CI-CD project % npm install', 'added 68 packages, and audited 69 packages in 3s', '15 packages are looking for funding', 'run `npm fund` for details', 'found 0 vulnerabilities', and '○ milo@Kyles-Air CI-CD project %' followed by a cursor.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh + ▢ 🗑️  
● milo@Kyles-Air CI-CD project % npm install  
added 68 packages, and audited 69 packages in 3s  
  
15 packages are looking for funding  
run `npm fund` for details  
  
found 0 vulnerabilities  
○ milo@Kyles-Air CI-CD project %
```

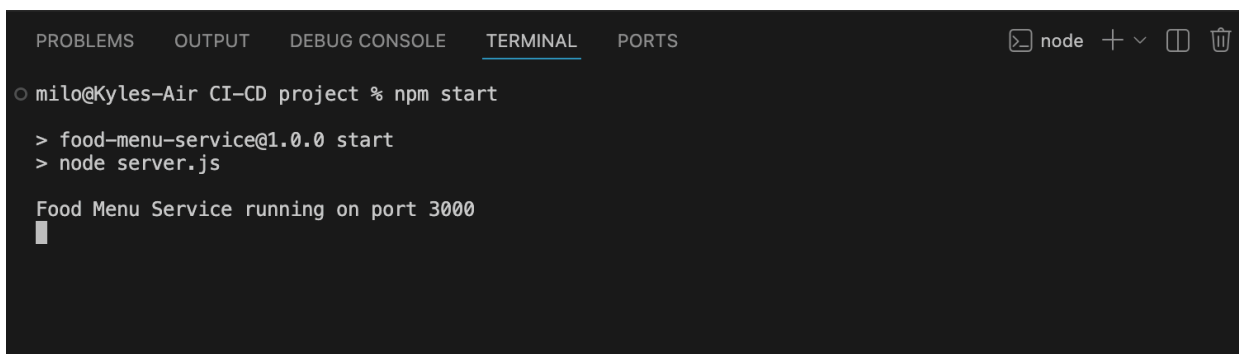
This installs Express and all other packages listed in `package.json`.

Step 4: Run the Application Locally

Start the application:

```
npm start
```

You should see: Food Menu Service running on port 3000

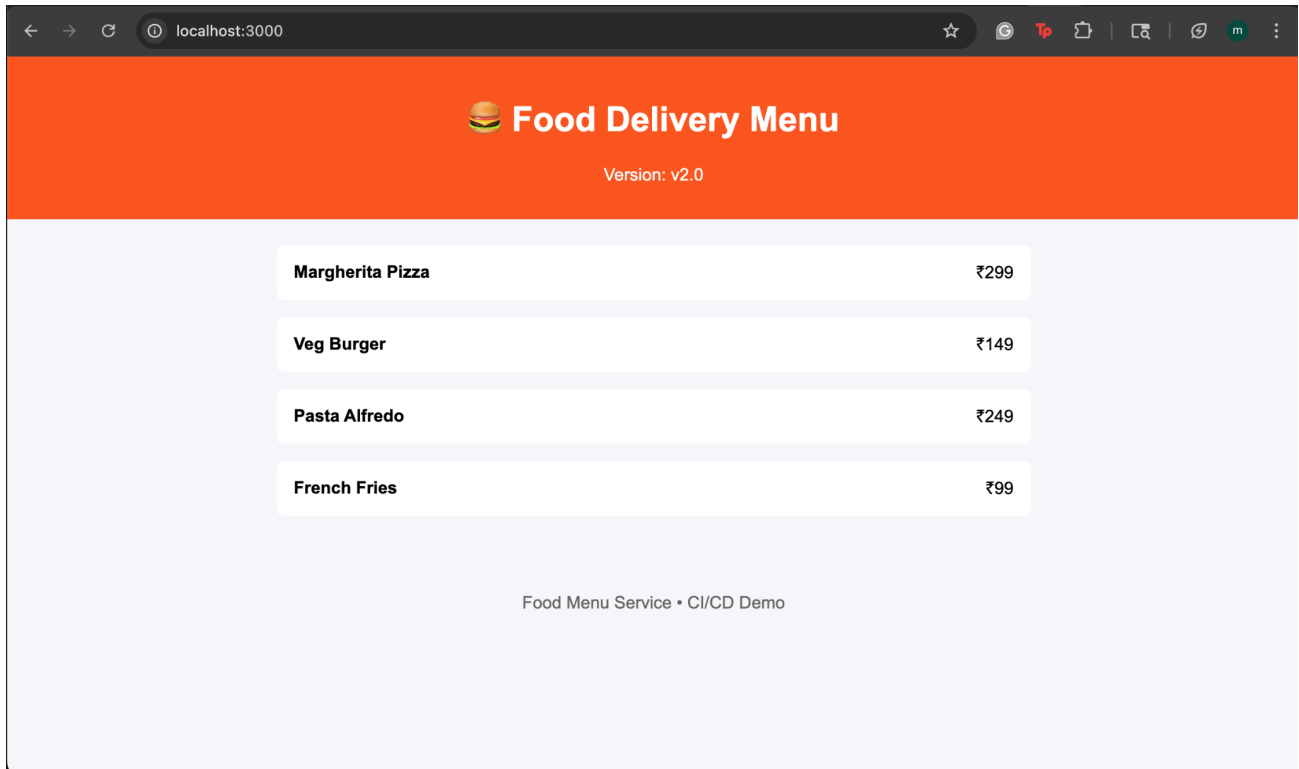
A screenshot of a terminal window with a dark background. The terminal shows the output of the 'npm start' command. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected), and 'PORTS'. On the right side of the terminal, there are icons for a shell (node), a plus sign, a dropdown arrow, a window icon, and a trash can icon. The output text in the terminal is: '○ milo@Kyles-Air CI-CD project % npm start', '> food-menu-service@1.0.0 start', '> node server.js', and 'Food Menu Service running on port 3000' followed by a cursor.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS node + ▢ 🗑️  
○ milo@Kyles-Air CI-CD project % npm start  
  
> food-menu-service@1.0.0 start  
> node server.js  
  
Food Menu Service running on port 3000  
█
```

Step 5: Access the Application UI

- Open a browser and navigate to:

```
http://localhost:3000
```



Step 6: Verify the Backend Endpoints

Test each API endpoint directly:

- **Menu API:** `http://localhost:3000/api/menu`

```
Pretty-print ☐
[{"id":1,"name":"Margherita Pizza","price":299},{"id":2,"name":"Veg Burger","price":149},{"id":3,"name":"Pasta Alfredo","price":249}, {"id":4,"name":"French Fries","price":99}]
```

- **Health check:** `http://localhost:3000/health`

```
Pretty-print ☐
{"status":"healthy"}
```

- **Version endpoint:** `http://localhost:3000/version`

```
Pretty-print ☐
{"version":"v2.0"}
```

Important: If the application does not run successfully at this stage, do not proceed to containerization. Resolve all issues locally first.

Phase 2: Containerize the Application Using Docker

Containerization packages the application and all its dependencies into a single portable unit. Once containerized, the application runs identically everywhere on a laptop, in a CI/CD pipeline, or in the cloud.

Step 1: Prerequisites

Ensure Docker is installed and running before proceeding.

- **Install Docker:** <https://docs.docker.com/get-docker/>

- **Confirm the installation:** `docker --version`

```
Last login: Sun Apr 12 23:48:03 on ttys000
[milo@Kyles-Air ~ % docker --version
Docker version 29.2.0, build 0b9d198
milo@Kyles-Air ~ % █
```

Step 2: Why Containerization?

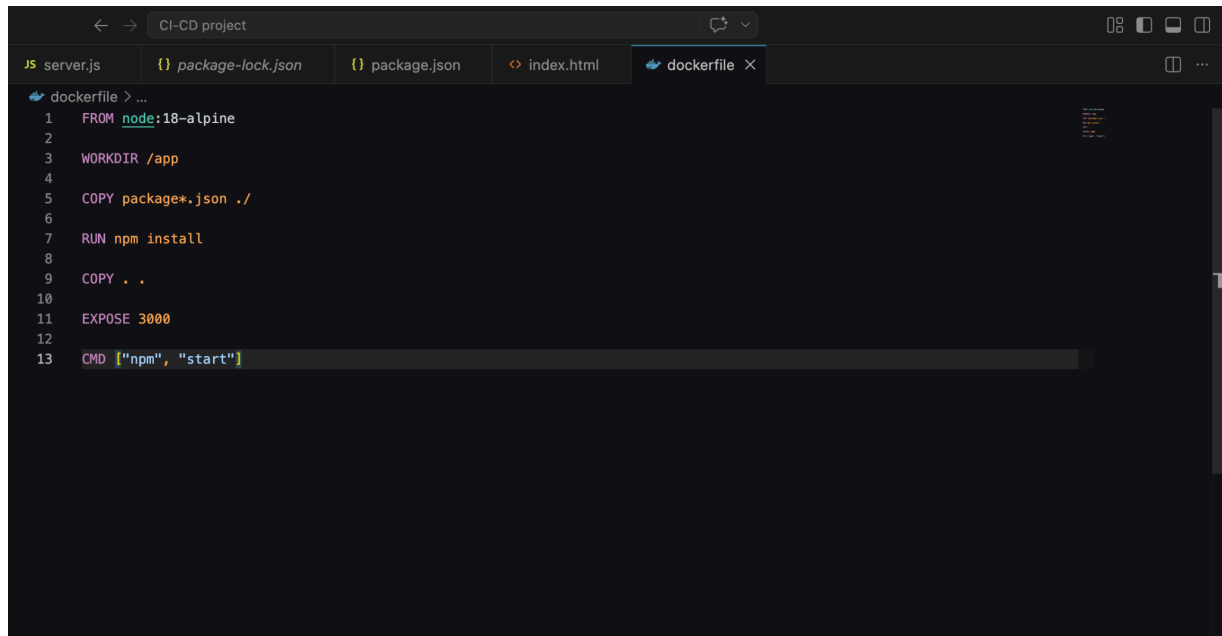
Applications often behave differently across machines due to OS differences, missing runtimes, or conflicting dependencies. This is the classic "it works on my machine" problem.

Containerization solves this by packaging the application alongside everything it needs creating a consistent, reproducible deployment unit.

Step 3: Create the Dockerfile

A Dockerfile is a set of build instructions. In the project root, create a file named `Dockerfile` (no extension) with the following content:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

A screenshot of a code editor interface for a 'CI-CD project'. The editor has a dark theme and a tab bar at the top with files: 'server.js', 'package-lock.json', 'package.json', 'index.html', and 'dockerfile'. The 'dockerfile' tab is active, showing a Dockerfile with 13 lines of code. The code starts with 'FROM node:18-alpine' and ends with 'CMD ["npm", "start"]'.

```
1 FROM node:18-alpine
2
3 WORKDIR /app
4
5 COPY package*.json ./
6
7 RUN npm install
8
9 COPY . .
10
11 EXPOSE 3000
12
13 CMD ["npm", "start"]
```

Step 4: Understanding the Dockerfile

Docker reads the Dockerfile line by line, from top to bottom:

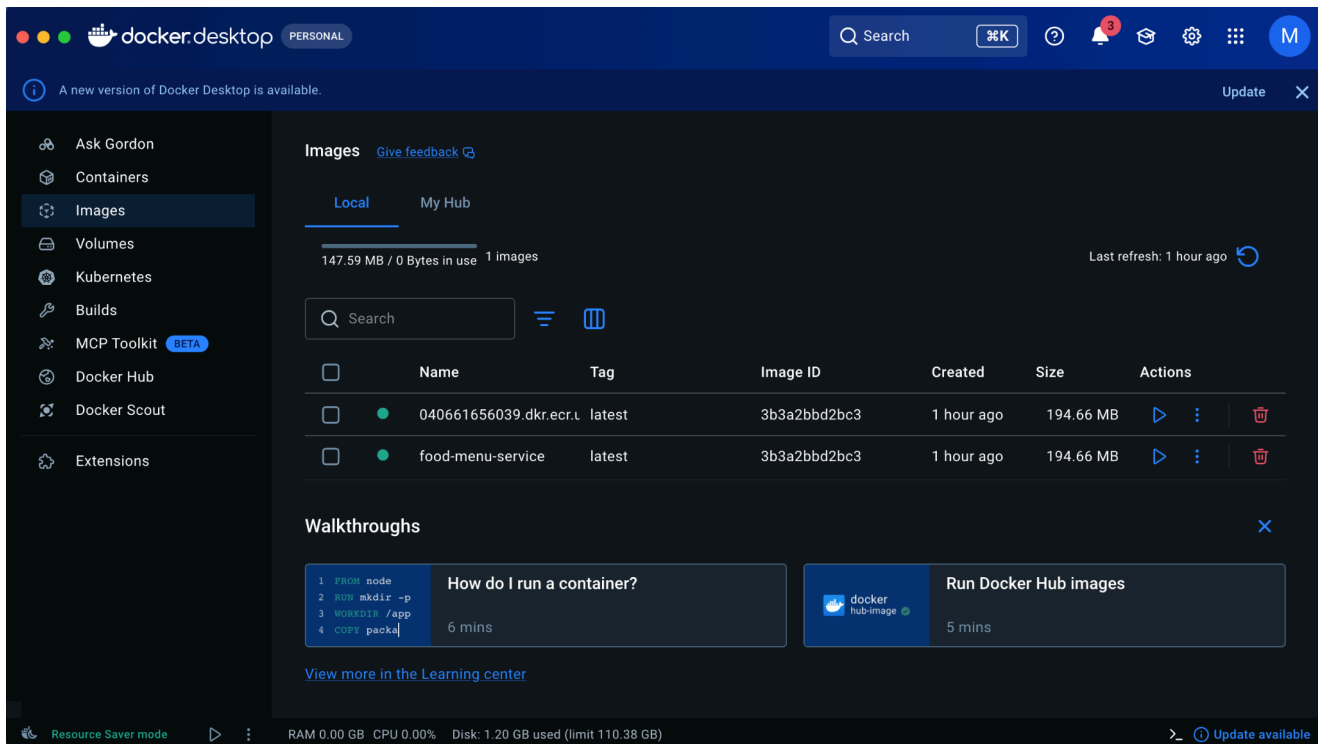
- `FROM node:18-alpine` - Sets the base image. Alpine Linux is lightweight and production-appropriate.
- `WORKDIR /app` - Sets the working directory inside the container.
- `COPY package*.json ./` - Copies dependency files first, enabling Docker layer caching for faster builds.
- `RUN npm install` - Installs application dependencies inside the container.
- `COPY . .` - Copies the full application source code into the container.
- `EXPOSE 3000` - Documents the port the application listens on.
- `CMD ["npm", "start"]` - Specifies the command to run when the container starts.

Step 5: Build the Docker Image

Important: AWS ECS Fargate requires the `linux/amd64` architecture. Build explicitly for this platform to ensure compatibility across Mac, Windows, and Linux:

```
docker build --platform linux/amd64 -t food-menu-service .
```

```
miloskyles-Air CI-CD project % docker build --platform linux/amd64 -t food-menu-service .
[+] Building 11.7s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring Dockerfile: 161B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc
=> => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B
=> => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB
=> => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4feea161b2b5a3d0e 40.01MB / 40.01MB
=> => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB
=> => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870
=> => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4feea161b2b5a3d0e
=> => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3
=> => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549
=> [internal] load build context
=> => transferring context: 2.35MB
=> [2/5] WORKDIR /app
=> [3/5] COPY package*.json ./
=> [4/5] RUN npm install
=> [5/5] COPY . .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:913648c3b8af225802c99439fbb009412e684daac32314432b363dcef55a85ec
=> exporting config sha256:a8de099c49521136bb50b10ac01b66f3459abe19d256c50751f9b1f85891f8b5
=> exporting attestation manifest sha256:17976f8efc8704ccd7eb71e24d74550cc7f183f9975301098774a9f381e72c64
=> exporting manifest list sha256:3b3a2bbd2bc349dcae7320c0bf5bcef8ce28c29d9fca2378e0fc78ffcc63a442
=> naming to docker.io/library/food-menu-service:latest
=> unpacking to docker.io/library/food-menu-service:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/jqk00rugz8cmj1ev5qq72p1o1
miloskyles-Air CI-CD project %
```



- After the build completes, verify the image exists:

```
docker images
```

```
miro@Kyles-Air CI-CD project % docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
food-menu-service:latest	3b3a2bbd2bc3	195MB	47.1MB	

```
miro@Kyles-Air CI-CD project %
```

Step 6: Run the Application Inside Docker

- Start the container and map the port to your local machine:

```
docker run -p 3000:3000 food-menu-service
```

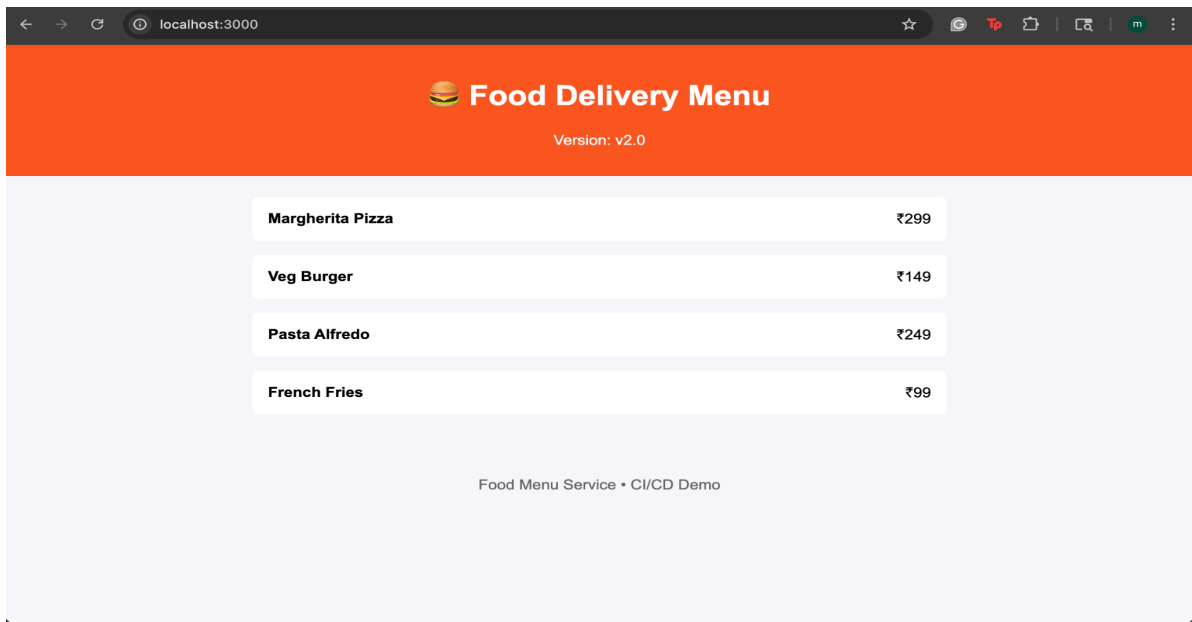
```
miro@Kyles-Air CI-CD project % docker run -p 3000:3000 food-menu-service
> food-menu-service@1.0.0 start
> node server.js

Food Menu Service running on port 3000
```

The `-p 3000:3000` flag maps port 3000 on your machine to port 3000 inside the container, allowing your browser to reach the containerized app.

Step 7: Verify the Containerized Application

- Open a browser and navigate to `http://localhost:3000`. You should see the same Food Delivery Menu UI as before, now running inside Docker.



Note: At this stage the container still runs on your local machine, so you access it via `localhost`. Once deployed to AWS, this address is replaced by a Load Balancer URL.

Phase 3: Push the Container Image to Amazon ECR

Amazon Elastic Container Registry (ECR) is AWS's managed container image registry. It serves as the centralized, secure store from which ECS pulls images during deployments.

Step 1: Prerequisites

Before proceeding, confirm the following are in place:

- An active AWS account
- AWS CLI installed on your system
- AWS CLI configured with valid credentials (`aws configure`)
- Docker installed and running

Step 2: Create an ECR Repository

The ECR repository is where your Docker images will be stored.

- Open the AWS Management Console
- Navigate to Amazon ECR
- Click Create repository
- **Enter a repository name:** `food-menu-service`
- Leave all other settings as default
- Click Create repository

Create private repository

General settings

Repository name
Enter a concise name. Repositories support namespaces, which you can use to group similar repositories.
040661656039.dkr.ecr.us-east-1.amazonaws.com/food-menu-service
17 out of 256 characters maximum (2 minimum). The name must start with a letter and can only contain lowercase letters, numbers, and special characters _./.

Image tag settings [Info](#)

Image tag mutability
Choose the tag mutability setting.

☒ **Mutable**
Image tags can be overwritten.

☐ **Immutable**
Image tags can't be overwritten.

Mutable tag exclusions
Tags that match these filters will be immutable (can't be overwritten). Using wildcards (*) will match zero or more image tag characters.
Add filter
Filters must only contain letters, numbers, and special characters (.,_,-). Each filter is limited to 128 characters, 2 wildcards (*), and you can add up to 5 filters in the exclusion list.

Encryption settings [Info](#)

The encryption settings for a repository can't be changed once the repository is created.

Encryption configuration

Once created, note the repository URI you will need it in subsequent steps.

Private repositories (1)

Search by repository substring

Repository name	Repository URI copied	Created at	Tag immutability	Encryption type
food-menu-service	040661656039.dkr.ecr.us-east-1.amazonaws.com/food-menu-service	April 13, 2026, 01:28:42 (UTC-04)	Mutable	AES-256

Step 3: Authenticate Docker with ECR

Run the following command, replacing <region> with your AWS region (e.g., us-east-1):

```
aws ecr get-login-password --region <region> | docker login \
  --username AWS --password-stdin \
  <aws_account_id>.dkr.ecr.<region>.amazonaws.com
```

A successful response shows: Login Succeeded

```

● milo@Kyles-Air CI-CD project % aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 040661656039.dkr.ecr.us-east-1.amazonaws.com

Login Succeeded
○ milo@Kyles-Air CI-CD project %

```

Step 4: Tag the Docker Image

Images must be tagged with the full ECR URI before pushing. First, confirm the image exists locally:

```
docker images
```

```

● milo@Kyles-Air CI-CD project % docker images

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
food-menu-service:latest             3b3a2bbd2bc3                     195MB       47.1MB       U
● milo@Kyles-Air CI-CD project % docker tag food-menu-service:latest 040661656039.dkr.ecr.us-east-1.amazonaws.com/food-menu-service:latest

○ milo@Kyles-Air CI-CD project %

```

Then tag the image:

```

docker tag food-menu-service:latest \
<aws_account_id>.dkr.ecr.<region>.amazonaws.com/food-menu-service:latest

```

Step 5: Push the Image to ECR

```

docker push
<aws_account_id>.dkr.ecr.<region>.amazonaws.com/food-menu-service:latest

```

```

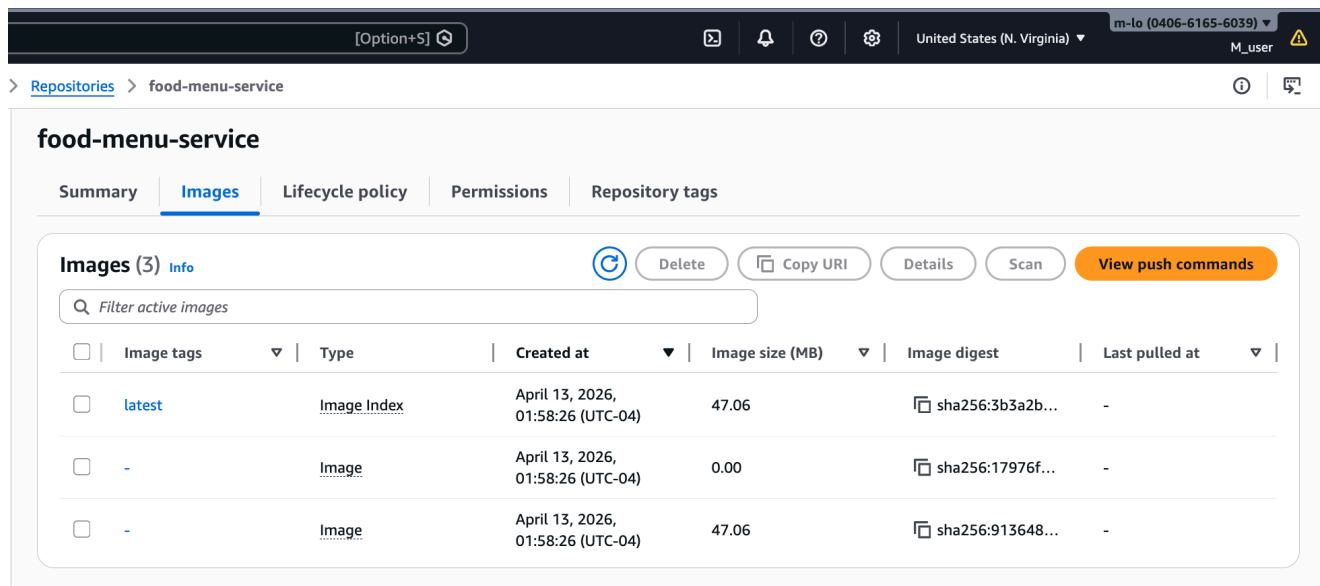
● milo@Kyles-Air CI-CD project % docker push 040661656039.dkr.ecr.us-east-1.amazonaws.com/food-menu-service:latest

The push refers to repository [040661656039.dkr.ecr.us-east-1.amazonaws.com/food-menu-service]
907dd890cf5a: Pushed
6da55bb23e3b: Pushed
f18232174bc9: Pushed
dd71dde834b5: Pushed
25ff2da83641: Pushed
df7ac7cc0499: Pushed
1e5a4c89cee5: Pushed
d5bc29cb2c1d: Pushed
5f64b045aaf3: Pushed
latest: digest: sha256:3b3a2bbd2bc349dcae7320c0bf5bcef8ce28c29d9fca2378e0fc78ffcc63a442 size: 856
○ milo@Kyles-Air CI-CD project %

```

Step 6: Verify the Image in ECR

- Open the Amazon ECR console, navigate to the `food-menu-service` repository, and confirm the image with the `latest` tag is listed.



The container image is now securely stored in AWS and ready to be deployed.

Phase 4: Deploy the App on Amazon ECS Fargate

Amazon ECS Fargate is a serverless container platform. AWS manages all the underlying infrastructure you simply define your container and ECS handles provisioning, scaling, and maintenance.

Step 1: Create the ECS Service-Linked IAM Role

ECS requires a service-linked IAM role to manage resources on your behalf. This step is required **only once per AWS account**.

Open the AWS Management Console → IAM → Roles → Create role

- Under Trusted entity type, select AWS service
- Under Use case, choose Elastic Container Service
- Select Elastic Container Service (ECS) and click Next
- Name the role `AWSServiceRoleForECS` and click Create role

Step 1: Select trusted entity

Select trusted entity [Info](#)

Trusted entity type

- ☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
Elastic Container Registry

Choose a use case for the specified service.

Use case

- ☒ **Elastic Container Registry - Replication**
Allows Amazon ECR to replicate images across Regions and accounts

[Cancel](#) [Next](#)

Step 2: Create an ECS Cluster

A cluster is a logical grouping for your containers. Create one named `food-menu-cluster`:

Open Amazon ECS → Clusters → Create cluster

- Enter cluster name: `food-menu-cluster`
- Under Infrastructure, select Fargate only
- Click Create

Amazon Elastic Container Service > Create cluster

food-menu-cluster
Cluster name must be 1 to 255 characters. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

► **Service Connect defaults - optional**

▼ **Infrastructure - advanced** [Info](#)
Configure the manner of obtaining compute resources that will be used to host your application.

Select a method of obtaining compute capacity
Your cluster is automatically configured for AWS Fargate (serverless), but you may choose to add Amazon EC2 instances (servers).

- ☒ **Fargate only**
Serverless - you don't think about creating or managing servers. Great for most common workloads.
- ☐ **Fargate and Managed Instances**
Managed instances - Amazon ECS will manage patching and scaling on your behalf while giving you configurability about the types of instances. Great for more advanced workloads.
- ☐ **Fargate and Self-managed instances**
Self-managed instances - you must ensure the instances are patched and scaled properly, and you have full control over the instances.

► **Monitoring - optional**
Configure observability, encryption, and logging options to maintain compliance and operational visibility of your container environment.

► **Encryption - optional**
Choose the KMS keys used by tasks running in this cluster to encrypt your storage.

► **Tags - optional** [Info](#)
Tags help you to identify and organize your clusters.

Step 3: Create an ECS Task Definition

A task definition describes how your container runs the image to use, CPU/memory allocation, port mappings, and environment settings.

In ECS, navigate to Task Definitions → Create new task definition

- Family name: `food-menu-task`
- Launch type: Fargate | OS: Linux
- Task CPU: 0.25 vCPU | Task memory: 0.5 GB
- Under Container definitions, enter container name: `food-menu-container`
- Image URI:
`<account>.dkr.ecr.<region>.amazonaws.com/food-menu-service:latest`
- Port mappings: Container port 3000, Protocol TCP

The screenshot displays the AWS Management Console interface for creating a new ECS task definition. The left sidebar shows the navigation menu with 'Amazon Elastic Container Service' selected. The main content area is titled 'Task definition configuration' and includes the following sections:

- Task definition family:** A text input field containing 'food-menu-task'. Below it, a note states: 'Up to 255 letters (uppercase and lowercase), numbers, hyphens, and underscores are allowed.'
- Infrastructure requirements:** A section with a dropdown arrow and an 'Info' link. It contains the following options:
 - Launch type:** A section with an 'Info' link. It states 'Selection of the launch type will change task definition parameters.' and has two options:
 - ☒ **AWS Fargate**: Serverless compute for containers.
 - ☐ **Managed Instances**: Use if you have specific hardware constraints, such as GPU accelerators, CPU instruction sets, or network-optimized hardware (offloads scaling, patching, and instance management to AWS).
 - ☐ **Amazon EC2 instances**: Self-managed infrastructure using Amazon EC2 instances.
- OS, Architecture, Network mode:** A section with an 'Info' link. It states 'Network mode is used for tasks and is dependent on the compute type selected.' and includes the following fields:
 - Operating system/Architecture:** A dropdown menu set to 'Linux/X86_64'.
 - Network mode:** A dropdown menu set to 'awsipc'.
 - Task size:** A section with an 'Info' link. It states 'Specify the amount of CPU and memory to reserve for your task.' and includes two fields:
 - CPU:** A dropdown menu set to '.25 vCPU'.
 - Memory:** A dropdown menu set to '.5 GB'.

Container - 1 [info](#) Essential container Remove

Container details
Specify a name, container image, and whether the container should be marked as essential. Each task definition must have at least one essential container.

Name
food-menu-container
Up to 255 letters (uppercase and lowercase), numbers, hyphens, and underscores are allowed.

Essential container
Yes

Image URI
040661656039.dkr.ecr.us-east-1.amazonaws.com/food-menu-service Browse ECR images
Up to 255 letters (uppercase and lowercase), numbers, hyphens, underscores, colons, periods, forward slashes, and number signs are allowed.

Private registry [info](#)
Store credentials in Secrets Manager, and then use the credentials to reference images in private registries.
☒ Private registry authentication

Port mappings [info](#)
Add port mappings to allow the container to access ports on the host to send or receive traffic. For port name, a default will be assigned if left blank.

Container port	Protocol	Port name	App protocol	
3000	TCP	container-port-protocol	HTTP	Remove

Add port mapping

Read only root file system [info](#)
When this parameter is turned on, the container is given read-only access to its root file system.
☐ Read only

Resource allocation limits - conditional [info](#)
Container-level CPU, GPU, and memory limits are different from task-level values. They define how much resources are allocated for the container. If container attempts to exceed the memory specified in hard limit, the container is terminated.

CPU	GPU	Memory hard limit	Memory soft limit

Step 4: Create an ECS Service

An ECS service keeps your task continuously running and handles restarts and scaling automatically. Navigate to your `food-menu-cluster` and click **Create service**:

- **Task definition:** `food-menu-task` (Latest revision)
- **Service name:** `food-menu-service`
- **Launch type:** Fargate
- **Desired tasks:** 1

Create service [info](#)

Service details

Task definition family
Select an existing task definition family. To create a new task definition, go to [Task definitions](#).

food-menu-task

Task definition revision [Latest](#)
Select the task definition revision from the 100 most recent entries, or enter a revision. Leave the field blank to use the latest revision.

Q 1

Service name
Assign a service name that is unique for this cluster.
food-menu-service
Up to 255 letters (uppercase and lowercase), numbers, underscores, and hyphens are allowed. Service names must be unique within a cluster.

Environment AWS Fargate

Existing cluster
food-menu-cluster

Compute configuration - advanced

Compute options [info](#)
To ensure task distribution across your compute types, use appropriate compute options.

☐ Capacity provider strategy
Specify a launch strategy to distribute your tasks across one or more capacity providers.

☒ Launch type
Launch tasks directly without the use of a capacity provider strategy.

Step 5: Configure the Security Group

Under Networking, create a new security group named `ecs-food-menu-sg`:

- **Inbound rule:** Custom TCP, Port 3000, Source: Anywhere
- **Ensure Public IP is set to Turned on**

The screenshot shows the AWS Management Console interface for creating a new service. The left sidebar contains navigation links for Amazon Elastic Container Service, including Express Mode, Clusters, Namespaces, Task definitions, Daemon task definitions, Account settings, AWS Batch, Amazon ECR, Repositories, Online learning workshop, Documentation, Discover products, and Subscriptions. The main content area is titled 'Create service' and shows the 'Security group' configuration. Under 'Security group', the option 'Create a new security group' is selected. The 'Security group details' section includes a 'Security group name' field with the value 'ecs-food-menu-sg' and a 'Security group description' field with the value 'Security group for food menu ECS service'. Below these fields, the 'Inbound rules for security groups' section shows a table with one rule: 'Custom TCP' type, 'TCP' protocol, '3000' port range, and 'Anywhere' source. The 'Public IP' section is set to 'Turned on'. At the bottom, the 'Service Connect' section is optional and allows for service-to-service communications.

Step 6: Attach an Application Load Balancer

The ALB routes internet traffic to your container and enables future zero-downtime deployments.

- **Enable Load balancing and select Application Load Balancer**
- **Load balancer name:** `food-menu-alb`
- **Listener:** Protocol HTTP, Port 80
- **Target group name:** `food-menu-tg`, Protocol HTTP, Port 80
- **Health check path:** `/`

Amazon Elastic Container Service

Express Mode
Clusters
Namespaces
Task definitions
Daemon task definitions **New**
Account settings

AWS Batch [↗](#)
Amazon ECR [↗](#)
Repositories [↗](#)

Online learning workshop [↗](#)
Documentation [↗](#)
Discover products [↗](#)
Subscriptions [↗](#)

[Tell us what you think](#)

Load balancer type [Info](#)
Specify the load balancer type to distribute incoming traffic across the tasks running in your service.

☒ **Application Load Balancer**
An Application Load Balancer makes routing decisions at the application layer (HTTP/HTTPS), supports path-based routing, and can route requests to one or more ports.

☐ **Network Load Balancer**
A Network Load Balancer makes routing decisions at the transport layer (TCP/UDP).

Container
The container and port to load balance the incoming traffic to

food-menu-container 3000:3000

Host port: Container port

Application Load Balancer
Specify whether to create a new load balancer or choose an existing one.

☒ **Create a new load balancer**
☐ Use an existing load balancer

Load balancer name
Assign a unique name for the load balancer.

food-menu-alb

Creates an internet-facing Application Load Balancer. To create an internal load balancer, use the [Amazon EC2 console](#).

Listener [Info](#)
Specify the port and protocol that the load balancer will listen for connection requests on.

☒ **Create new listener**
☐ Use an existing listener
You need to select an existing load balancer.

Port: 80

Protocol: HTTP

Target group [Info](#)
Specify whether to create a new target group or choose an existing one that the load balancer will use to route requests to the tasks in your service.

Edit inbound rules [Info](#)
Inbound rules control the incoming traffic that's allowed to reach the instance.

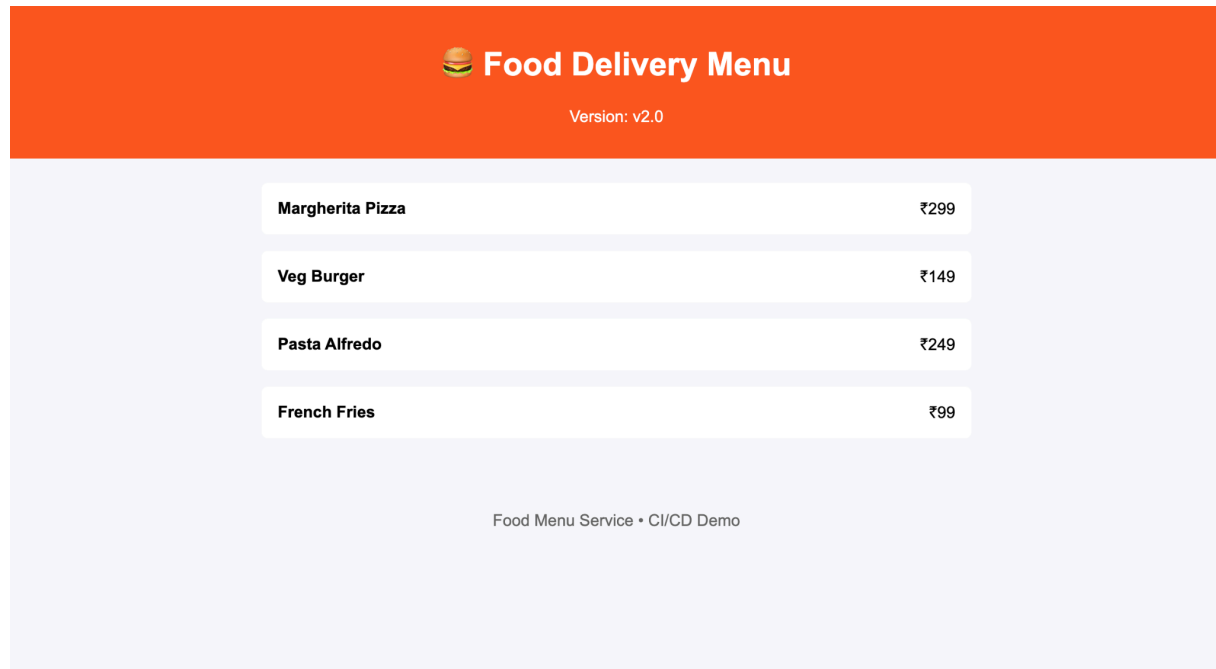
Security group rule ID	Type	Protocol	Port range	Source	Description - optional	
sg-r0f1f612936a5dfb78	HTTP	TCP	80	Custom	0.0.0.0/0	Delete
sg-r06e7bc4918d0208c2	Custom TCP	TCP	3000	Custom	0.0.0.0/0	Delete
sg-r0d702ac71e2607729	Custom TCP	TCP	3000	Custom	0.0.0.0/0	Delete

[Add rule](#)

Click Create service. ECS will provision the load balancer, register the task as a target, and start the container. This may take a few minutes.

Step 7: Verify the Deployment

- Once the service is running:
- Go to EC2 → Load Balancers
- Open food-menu-alb and copy the DNS name
- Paste the DNS name into your browser



The Food Delivery Menu UI loads successfully the container is running on Fargate, traffic is routed through the load balancer, and the application is publicly accessible.

Phase 5: Automate Deployments with GitHub Actions

With the application running manually on ECS, the final step is to automate the entire build-push-deploy cycle. From this point forward, GitHub becomes the single source of truth every push to the main branch triggers a full deployment automatically.

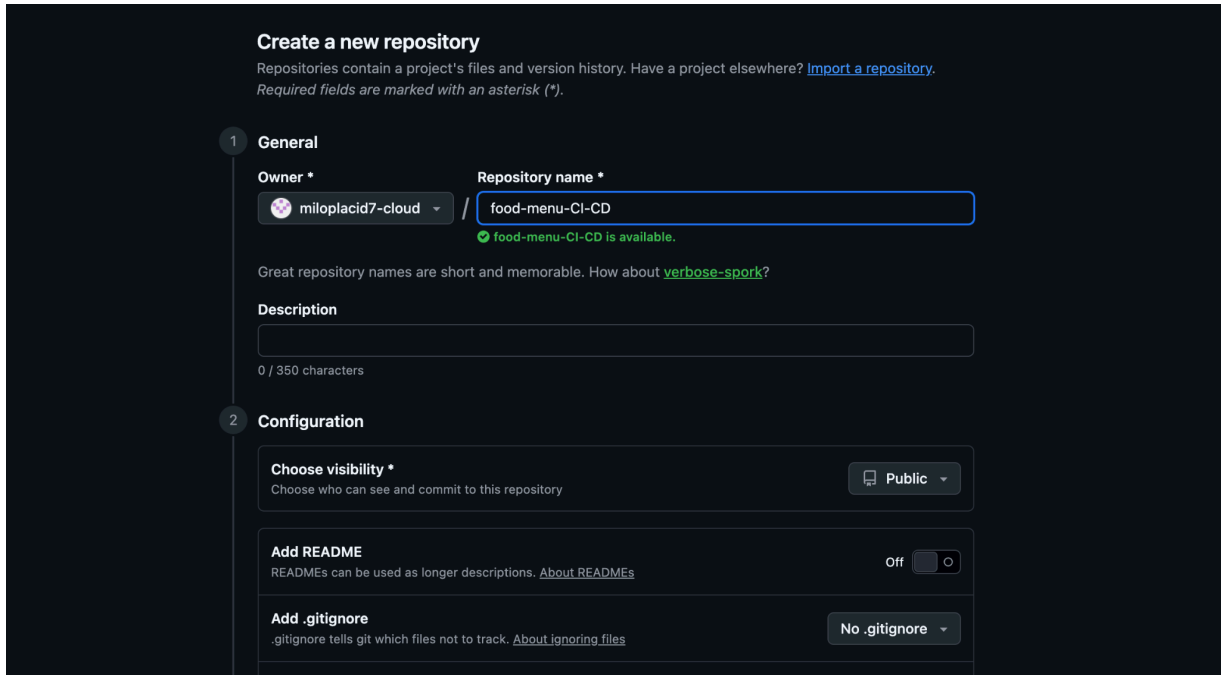
How the Pipeline Works:

- On every push to the main branch, GitHub Actions will:
- Check out the latest source code
- Build a new Docker image
- Push the image to Amazon ECR
- Force a new ECS deployment with the updated image
- Automatically replace the running container with zero downtime

Step 1: Push the Project to GitHub

The pipeline requires the application code to be stored in GitHub.

- Create a new repository on GitHub (do not initialize with README)



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * miloplad7-cloud

Repository name * food-menu-CI-CD
✔ food-menu-CI-CD is available.

Great repository names are short and memorable. How about [verbose-spork?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#) Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

- From the project root directory, run:

```
git init
git add .
git commit -m "Initial commit: Food Menu Service"
git branch -M main
git remote add origin <YOUR_GITHUB_REPO_URL>.git
git push -u origin main
```

```
● milo@Kyles-Air CI-CD project % git init
  Initialized empty Git repository in /Users/milo/CI-CD project/.git/
● milo@Kyles-Air CI-CD project % git add .
● milo@Kyles-Air CI-CD project % git commit -m "initail commit:Food Menu Service"
[main (root-commit) 0983e69] initail commit:Food Menu Service
 621 files changed, 67587 insertions(+)
  create mode 100644 dockerfile
  create mode 120000 node_modules/.bin/mime
  create mode 100644 node_modules/.package-lock.json
  create mode 100644 node_modules/accepts/HISTORY.md
  create mode 100644 node_modules/accepts/LICENSE
  create mode 100644 node_modules/accepts/README.md
  create mode 100644 node_modules/accepts/index.js
  create mode 100644 node_modules/accepts/package.json
```

```

● milo@Kyles-Air CI-CD project % git branch -M main
● milo@Kyles-Air CI-CD project % git remote add origin https://github.com/miloplacid7-cloud/food-menu-CI-CD.git
● milo@Kyles-Air CI-CD project % git push -u origin main
Enumerating objects: 716, done.
Counting objects: 100% (716/716), done.
Delta compression using up to 4 threads
Compressing objects: 100% (660/660), done.
Writing objects: 100% (716/716), 718.48 KiB | 3.22 MiB/s, done.
Total 716 (delta 129), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (129/129), done.
To https://github.com/miloplacid7-cloud/food-menu-CI-CD.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
○ milo@Kyles-Air CI-CD project %

```

Step 2: Create an IAM User for GitHub Actions

- GitHub Actions needs AWS permissions to push images to ECR and update the ECS service.
- Open AWS Console → IAM → Users → Create user
- Username: github-actions

Specify user details

User details

User name

github-actions

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☒ **Provide user access to the AWS Management Console - optional**
In addition to console access, users with `SignInLocalDevelopmentAccess` permissions can use the same console credentials for programmatic access without the need for access keys.

Console password

☐ **Autogenerated password**
You can view the password after you create the user.

☒ **Custom password**
Enter a custom password for the user.

- Must be at least 8 characters long
- Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # \$ % ^ & * () _ + - (hyphen) = [] { } | ' "

☐ Show password

☐ **Users must create a new password at next sign-in - Recommended**
Users automatically get the [IAMUserChangePassword](#) policy to allow them to change their own password.

Info If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)

- Attach the following policies:
 - AmazonEC2ContainerRegistryFullAccess
 - AmazonECS_FullAccess

Permissions summary			< 1 >
Name ↗	Type	Used as	
AmazonEC2ContainerRegistryFullAccess	AWS managed	Permissions policy	
AmazonECS_FullAccess	AWS managed	Permissions policy	

- Click Create user

Step 3: Create an Access Key

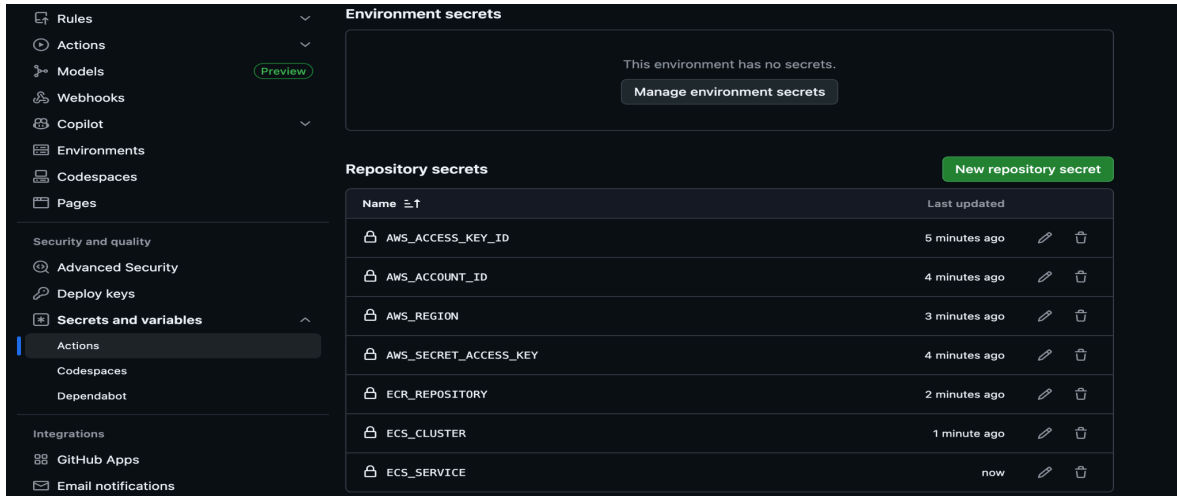
- Open the newly created `github-actions` user → Security credentials → Access keys → Create access key → CLI.

Important: Copy both the Access Key ID and Secret Access Key immediately. The secret key cannot be viewed again after this screen.

Step 4: Add GitHub Repository Secrets

Navigate to your GitHub repository → Settings → Secrets and variables → Actions, and add:

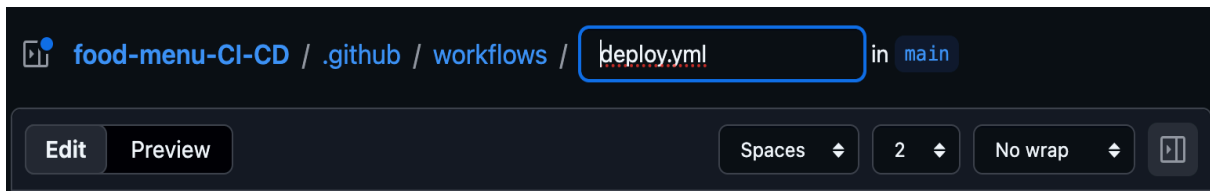
- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`
- `AWS_ACCOUNT_ID`
- `AWS_REGION` → `us-east-1`
- `ECR_REPOSITORY` → `food-menu-service`
- `ECS_CLUSTER` → `food-menu-cluster`
- `ECS_SERVICE` → `food-menu-service`



Step 5: Create the GitHub Actions Workflow

- In your repository, create the file `.github/workflows/deploy.yml`:

Note: GitHub Actions only executes workflows placed inside the `.github/workflows/` directory. If this folder does not exist, the pipeline will not trigger.



The workflow file defines the following pipeline steps:

- **Checkout source code** - pulls the repository into the GitHub runner
- **Configure AWS credentials** - authenticates using the IAM secrets stored in GitHub
- **Log in to Amazon ECR** - authenticates Docker with the registry
- **Build Docker image** - builds for linux/amd64 architecture
- **Tag and push image to ECR** - uploads the new image to the registry
- **Update ECS service** - forces ECS to deploy the new container version

```
food-menu-CI-CD / .github / workflows / deploy.yml in main
Edit Preview Spaces 2 No wrap
1 name: Deploy to Amazon ECS
2
3 on:
4   push:
5     branches:
6       - main
7
8 jobs:
9   deploy:
10    runs-on: ubuntu-latest
11
12    steps:
13      - name: Checkout source code
14        uses: actions/checkout@v4
15
16      - name: Configure AWS credentials
17        uses: aws-actions/configure-aws-credentials@v4
18        with:
19          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
20          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
21          aws-region: ${ secrets.AWS_REGION }
22
23      - name: Login to Amazon ECR
```

Step 6: Trigger and Observe the Pipeline

- Commit and push the `deploy.yml` file to the main branch, then open your GitHub repository and navigate to Actions. Select the latest workflow run to watch each step execute in real time.

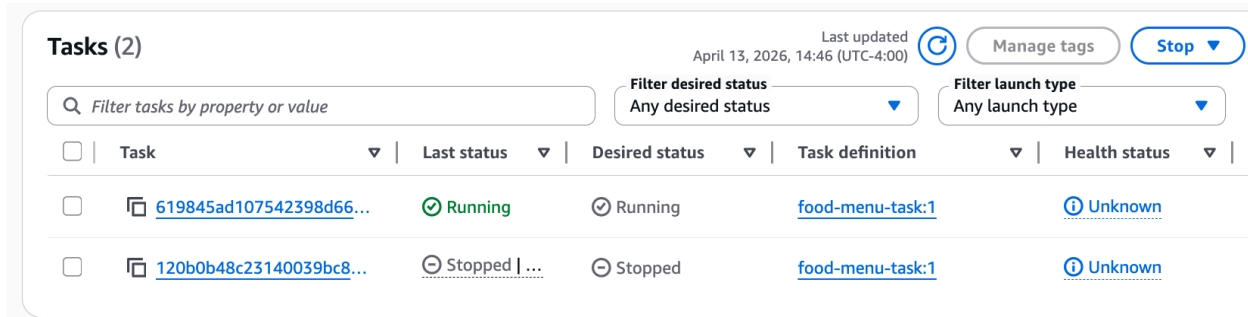
The screenshot displays the GitHub Actions interface for a workflow named 'Deploy to Amazon ECS'. The top navigation bar includes links for Code, Issues, Pull requests, Actions (selected), Projects, Wiki, Security and quality, Insights, and Settings. Below the navigation bar, the workflow is identified as 'Update deploy.yml #3' with a green checkmark icon. A 'Re-run all jobs' button is visible. The left sidebar shows a list of jobs: 'Summary', 'All jobs', and 'deploy' (selected). The main content area shows the 'deploy' job status as 'succeeded 27 minutes ago in 16s'. Below this, a list of steps is shown with their durations:

Step	Duration
Set up job	1s
Checkout source code	0s
Configure AWS credentials	0s
Login to Amazon ECR	1s
Build Docker image	7s
Tag Docker image	0s
Push Docker image to Amazon ECR	1s
Update ECS service	3s

Once the workflow completes successfully, GitHub Actions will have built a new Docker image, pushed it to ECR, and triggered a fresh ECS deployment automatically.

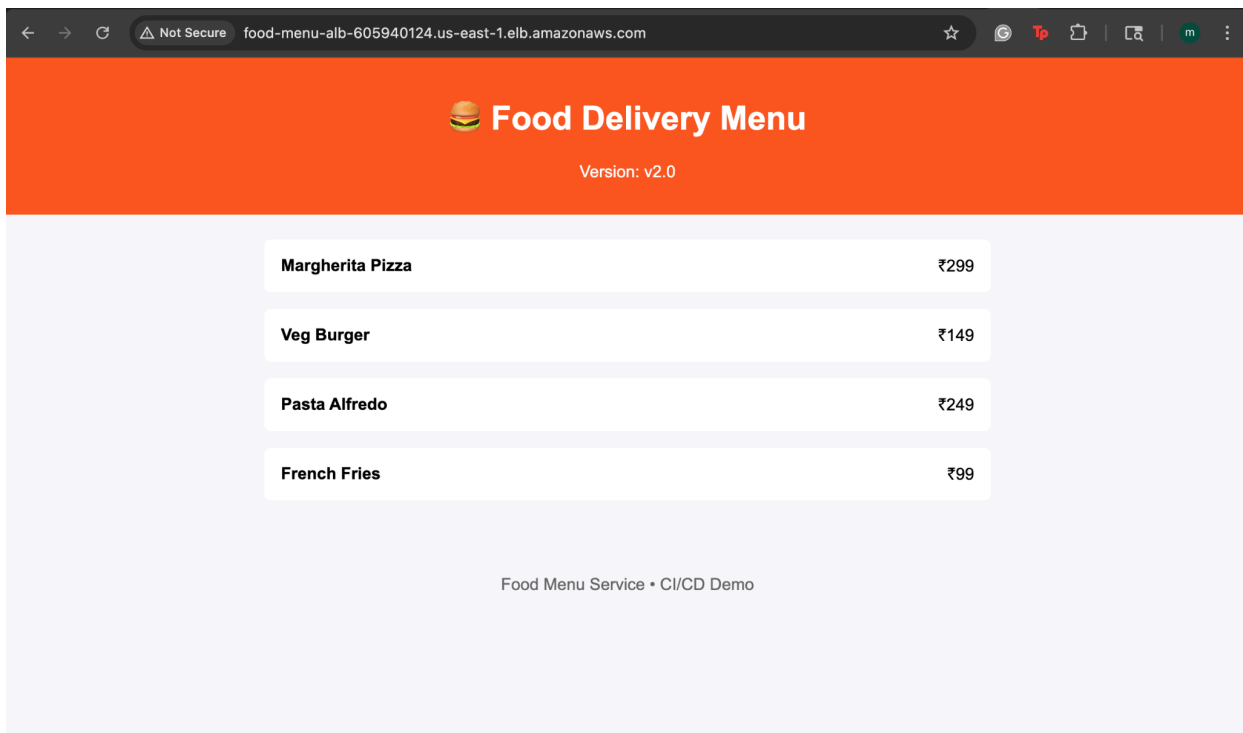
Step 7: Verify the CI/CD Deployment

- Open Amazon ECS and confirm a new task revision is running:



Tasks (2)		Last updated April 13, 2026, 14:46 (UTC-4:00)		Manage tags	Stop
Filter tasks by property or value		Filter desired status Any desired status		Filter launch type Any launch type	
☐	Task	Last status	Desired status	Task definition	Health status
☐	619845ad107542398d66...	Running	Running	food-menu-task:1	Unknown
☐	120b0b48c23140039bc8...	Stopped ...	Stopped	food-menu-task:1	Unknown

- Open the Application Load Balancer DNS in your browser and confirm the updated version is visible:



The CI/CD pipeline is working correctly. The fully automated pipeline for a containerized application on AWS is complete.

Conclusion

By completing this project, you have implemented a production-grade DevOps workflow from end to end. Specifically, you have:

- **Built and containerized** a full-stack Node.js web application using Docker
- **Stored and managed container images** securely using Amazon ECR
- **Deployed containers serverlessly** using Amazon ECS Fargate no server management required
- **Exposed the application to the internet** through an Application Load Balancer
- **Configured networking and security groups** for safe, controlled traffic flow
- **Automated the full CI/CD pipeline** using GitHub Actions for continuous integration and delivery
- **Implemented zero-downtime deployments** where every code push triggers an automated production release

This project reflects a real-world SaaS deployment pattern. **This kind of pipeline to deliver faster releases, eliminate manual errors, and maintain consistent, reliable environments across the software development lifecycle.**

Cleanup Steps

Important: AWS resources incur charges even when idle. Complete the following steps after finishing this project to avoid unexpected costs.

Delete ECS Service and Cluster

- Go to Amazon ECS → Clusters → `food-menu-cluster`
- Delete the `food-menu-service` and wait for all running tasks to stop
- Delete the cluster

Delete the Application Load Balancer

- Go to EC2 Console → Load Balancers
- Select the `food-menu-alb` and click Delete

Delete Target Groups

- Go to EC2 Console → Target Groups
- Delete all target groups associated with `food-menu-service`

Delete the ECR Repository

- Go to Amazon ECR → Repositories → `food-menu-service`
- Delete the repository and all images inside it

Remove IAM User

- Go to IAM → Users → delete the `github-actions` user

Delete Security Groups

- Go to EC2 → Security Groups
- Delete any security groups created for this project
- Ensure they are not attached to other resources before deleting

Notes and Best Practices

- **Always validate locally before deploying.** If the application does not run on your machine, it will not run reliably in the cloud.
- **Use consistent image tagging** - Stick to one image name and tagging strategy (e.g., `latest` or commit SHA) to avoid confusion during deployments.
- **Restrict container access** - Allow inbound traffic only from the load balancer security group, not directly from the internet.
- **Protect CI/CD credentials** - Always store AWS access keys in GitHub Secrets never commit them to source code.
- **Clean up resources promptly** - AWS charges for running resources. Delete all project infrastructure after testing to avoid unexpected billing.